TECHNICAL REFERENCE V4.2

Shokunin 職人
AI Engineering Ecosystem

37 skills · ChromaDB persistent memory · 3 MCP servers · 4 subagents · Cross-platform (Windows + Linux) · CI/CD · Zero cost, open source

Elias Oulkadi

Version 4.2 · May 2026
github.com/EliasOulkadi/shokunin

Contents

Executive Summary

Shokunin (職人, Japanese for artisan) is an open source ecosystem that transforms OpenCode into a production-grade AI engineering workstation. 37 domain-specific skills, persistent ChromaDB memory, and cross-platform support eliminate the blank-slate problem that plagues every AI coding session.

Instead of starting every session with zero context, Shokunin provides a complete engineering environment: domain expertise through auto-activating skills, persistent context across sessions through a three-layer memory system, direct system access through MCP servers, and automated maintenance through platform-native scheduling.

> One command install:
> Windows: `irm https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex`
> Linux: `bash <(curl -sL https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)`

Key Takeaways

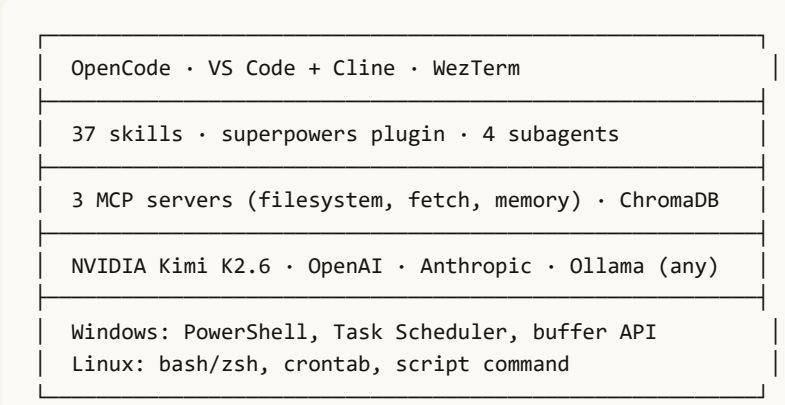
- 37 curated skills — infrastructure, backend, frontend, mobile, content, productivity. Each with procedural workflows, error handling tables, and production checklists.
- Three-layer memory — agent-driven saves, parallel checkpoint timer (every 5 min), console buffer capture on exit. ChromaDB semantic search with markdown fallback.
- Cross-platform — Windows and Linux share the same Python core, skills, MCP servers, and configs. Platform-specific wrappers only for terminal capture and scheduling.
- Zero cost — NVIDIA free API, open source MIT license, no subscriptions, no cloud, no telemetry.

02 · ARCHITECTURE

Architecture & Design

Five independent layers that each serve a specific role. Every layer can be replaced, extended, or run on a different operating system without affecting the others.

System Layers



Design Principles

1. Local-first — all data stays on your machine. No cloud, no telemetry, no subscriptions.
2. Zero cost — NVIDIA free API tier, open source MIT license.
3. Auto-activating — skills match your task description automatically.
4. Persistent memory — session context survives restarts, reboots, and power outages.
5. Cross-platform — shared Python core with platform-specific wrappers.
6. Idempotent installer — safe to re-run, creates backups before changes.

Data Flow

When you describe a task, OpenCode reads all skill descriptions (~100 tokens each). The agent matches your words against descriptions and loads the matching SKILL.md into context. MCP servers provide filesystem access, web fetching, and memory. At session end, the wrapper captures terminal output and saves to ChromaDB + markdown.

The 37 Skills

Each skill is a SKILL.md file with trigger-optimized descriptions, procedural workflows, error handling tables, production checklists, anti-patterns, and cited sources. They auto-activate when the agent matches your description.

Infrastructure (5)

Skill	What it does	Example trigger
docker	Multi-stage builds, BuildKit cache, compose, vulnerability scanning	"Containerize my app"
kubernetes	Gateway API, service mesh, HPA, PDB, pod debugging	"Deploy to K8s"
terraform	Stacks, test framework, pre/postconditions, state surgery	"Set up AWS"
ci-cd	GitHub Actions, GitLab CI, CircleCI, OIDC, canary	"Add CI pipeline"
db-admin	PostgreSQL backup, replication, WAL archiving, PITR	"Backup database"

Backend (4)

Skill	What it does	Example trigger
auth-architect	OAuth2 + PKCE, WebAuthn, JWT rotation, RBAC, OWASP	"Add login"
api-forge	REST/GraphQL, OpenAPI 3.1, webhooks, idempotency	"Design an API"
db-sculptor	Index strategy, EXPLAIN ANALYZE, migrations, Prisma/Drizzle	"Optimize query"
error-handler	OpenTelemetry, retry with jitter, circuit breaker, error budgets	"Add observability"

Frontend (5)

Skill	What it does	Example trigger
component-forge	React/Vue/Svelte, TypeScript strict, WCAG 2.2	"Create a button"
responsive-engine	Container Queries, clamp(), :has(), subgrid	"Make responsive"
motion-craft	WAAPI, Scroll-Driven Animations, FLIP, spring physics	"Add animation"
landing-craft	CRO frameworks, A/B testing, Core Web Vitals	"Design landing page"
ui-ux-pro-max	DB of UI patterns, palettes, font pairings, 13 stacks	"Pick a palette"

Mobile (2)

Skill	What it does	Example trigger
flutter	Riverpod, Impeller, Dart 3.7+, Pigeon, Clean Architecture	"Build Flutter app"
react-native	Expo Router, FlashList, Reanimated 4, Hermes	"Build RN app"

Quality (2)

Skill	What it does	Example trigger
test-commander	Testing Trophy, Vitest, MSW, Playwright E2E, visual regression	"Add tests"
performance-profiler	Lighthouse, Core Web Vitals, bundle analysis, caching	"Audit performance"

Content & Business (6)

Skill	What it does	Example trigger
communication	Emails, SBI/BID feedback, meeting notes, difficult conversations	"Write an email"
content-marketing	AIDA/PAS/BAB, headline formulas, Cialdini, copywriting	"Write blog post"
business-proposals	Cold email sequences, GBB pricing, SOWs, pitch decks	"Create proposal"
seo-geo	SEO + GEO 2026, llms.txt, structured data, AI Overviews	"Optimize for search"
translate-craft	8 languages, cultural adaptation, ICU syntax, RTL	"Translate to JP"
kami	Professional PDFs, parchment design, 8 templates	"Create PDF"

Productivity (11)

Skill	What it does	Example trigger
git-workflow	Branch/commit/PR/rebase automation	"Create branch"
windows-powershell	System info, disk cleanup, dev tools	"Check health"
strategy	Brainstorming, RICE, weighted scoring	"Decide approach"
design	Brand guidelines, design tokens, briefs	"Design system"
documentation	READMEs, API docs, changelogs, KBs	"Write docs"
runbook-gen	Incident runbooks, post-mortems, war room	"Site is down"
finance	5-pillar planning, tax, FIRE, insurance	"Plan finances"
legal-counsel	GDPR, EU AI Act, CCPA, HIPAA, DMCA	"GDPR requirements"
whendone-plus	Desktop notifications for long commands	"Notify when done"
memory	ChromaDB persistent memory management	"Search memory"
chromadb	View, search, delete, backup, reset vector DB	"Show stats"

Memory System v4.2

Three layers ensure no context is lost between sessions. ChromaDB provides semantic search; a parallel checkpoint timer saves every 5 minutes; the wrapper captures terminal output on exit.

Architecture

```
┌─────────────────────────────────────────────────────┐
│              MEMORY SYSTEM v4.2                     │
├─────────────────────────────────────────────────────┤
│  LAYER 1: Agent-driven saves (CLAUDE.md)            │
│  ├─ search_context at session start                 │
│  ├─ store_context every 3-5 turns (type=checkpoint) │
│  ├─ store_context on decisions/files/commands       │
│  └─ store_context at session end (type=session_end) │
│                                                     │
│  LAYER 2: Parallel checkpoint timer (every 5 min)   │
│  ├─ Windows: .NET Timer thread                      │
│  ├─ Linux: bash background sleep loop               │
│  └─ Saves regardless of agent compliance            │
│                                                     │
│  LAYER 3: Terminal capture on exit                  │
│  ├─ Windows: GetBufferContents (5000 lines)         │
│  ├─ Linux: script command (unlimited)               │
│  └─ Strips ANSI, saves raw + parsed + ChromaDB      │
│                                                     │
│  STORAGE: ChromaDB (semantic) + .md (grep) + .log (raw) │
└─────────────────────────────────────────────────────┘
```

Data Format

Every entry uses typed metadata for precise filtering:

```
{
  "type": "decision | file | command | preference | checkpoint | session_end",
  "timestamp": "2026-05-14T10:30:00Z",
  "project": "my-project",
  "tags": ["auth", "jwt", "decision"],
  "session_id": "session-20260514-103000-A7F3",
  "text": "Decided to use asymmetric JWT signing (RS256)"
}
```

Workflow: Finding Past Context

Situation: You start a new session and need to remember past auth decisions.

Automatic flow:

1. Agent reads CLAUDE.md: "search_context at session start (mandatory)"
2. Agent calls `search_context("authentication auth jwt", "my-project")`
3. ChromaDB returns top 5 most relevant entries from past sessions
4. Agent presents: "Found 3 sessions about auth. Key decisions: RS256 signing, refresh token rotation"

Storage

What	Path	Size
ChromaDB vector DB	~/.shokunin/memory/chroma_db/	400 KB + ~1 KB/entry
Session logs (raw)	~/.shokunin/memory/sessions/*.log	1-10 KB/session
Session summaries	~/.shokunin/memory/sessions/*.md	0.5-3 KB/session
Active session	~/.shokunin/current-session.json	~200 bytes

Environment Variables

Variable	Value
SHOKUNIN_SESSION_ID	Current session UUID
SHOKUNIN_PROJECT	Working directory
SHOKUNIN_MCP_HEALTHY	"1" if MCP active
SHOKUNIN_LAST_SESSION	Previous session ID

MCP Servers

MCP servers give OpenCode direct system access. They start on demand via opencode.json, consume no resources when idle, and work identically on Windows and Linux.

Server	Tools	Purpose
filesystem	read, write, edit, search, list	File operations with path validation
fetch	fetch, fetch_json	Web pages and API calls
memory	store_context, search_context, get_session_summary	ChromaDB with type filtering

Memory MCP Server

150 lines of Python, JSON-RPC 2.0 over stdin/stdout. Logs all operations.

```
# store_context
echo '{"jsonrpc":"2.0","id":2,"method":"tools/call",
  "params":{"name":"store_context",
    "arguments":{"text":"Auth uses RS256","session_id":"s-123",
      "type":"decision","tags":["auth"]}}}' | python mcp-server.py
```

Subagents

Four specialized subagents run in parallel with their own models. They handle code review and debugging without blocking the main conversation.

Subagent	Model	Fallback	Purpose
code-review	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Bug + security review
code-review-local	Ollama Qwen3 14B	—	Offline code review
debugger	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Root cause analysis
debugger-local	Ollama Qwen3 14B	—	Offline debugging

Cross-Platform: Windows & Linux

The same skills, memory, MCP servers, and subagents work on both OS. Only the wrappers and installers differ.

Shared (identical)

Component	Windows	Linux
chroma-helper.py	✅	✅
mcp-server.py	✅	✅
37 skills	✅	✅
CLAUDE.md / AGENTS.md	✅	✅
opencode.json	✅	✅
4 subagents	✅	✅

Platform-Specific

Capability	Windows	Linux
Terminal capture	Buffer API (5000 lines)	script command (unlimited)
Wrapper	run-opencode.ps1	run-opencode.sh
Installer	install.ps1	install.sh
Profile shadow	PowerShell $PROFILE	.bashrc/.zshrc
Scheduling	Task Scheduler	crontab
Healthcheck	memory-healthcheck.ps1	memory-healthcheck.sh
Maintenance	weekly-healthcheck.ps1	weekly-maintenance.sh

Install

```
Windows: irm https://.../install.ps1 | iex
Linux:   bash <(curl -sL https://.../install.sh)
```

Both are idempotent: backup before write, skip existing, safe to re-run.

> Linux advantage: The `script` command captures all terminal output without the 5000-line limit that Windows buffer API has. For long sessions, Linux captures more context.

Workflow Examples

Real scenarios showing how the ecosystem works end-to-end. Describe your task in natural language — skills auto-activate.

Full-Stack Feature: Authentication

User: "Add Google OAuth login to my Express app"

1. auth-architect → OAuth2 + PKCE flow, JWT rotation, session management
2. db-sculptor → User schema with Prisma, migration, indexes
3. error-handler → Structured error handling for auth routes
4. test-commander → Integration tests with mocked OAuth
5. Memory checkpoint: "Added Google OAuth with PKCE, RS256 signing"

Deployment: Docker + Kubernetes

User: "Dockerize and deploy to production K8s"

1. docker → Multi-stage Dockerfile, BuildKit cache, .dockerignore, compose
2. kubernetes → Deployment, Service, Ingress, HPA, PDB
3. ci-cd → GitHub Actions: build → test → deploy → healthcheck → rollback
4. db-admin → PostgreSQL backup strategy, connection pooling

Debugging: Production Incident

User: "API returning 502 errors"

1. runbook-gen → Incident template, severity assessment
2. error-handler → Log analysis, root cause suggestions
3. debugger subagent → Deep stack trace analysis
4. memory → Search past sessions for similar incidents

Content: Blog Post + PDF

User: "Write a blog and export as PDF"

1. content-marketing → AIDA framework, headline, SEO structure
2. seo-geo → Structured data, llms.txt, AI Overview optimization
3. kami → Parchment design HTML → professional PDF

CI/CD & Quality

GitHub Actions validates every component on every push. Four parallel jobs complete in ~2 minutes.

Pipeline

Job	OS	Validates
test-python	Ubuntu	ChromaDB import, chroma-helper save/search, MCP tools, MCP store/search
test-scripts	Windows	PowerShell syntax, chroma-helper on Windows
test-linux	Ubuntu	Bash syntax, cross-platform paths, install.sh parsing
validate-config	Ubuntu	JSON validity, SKILL.md required fields, CLAUDE.md memory section

QUALITY COMMITMENTS

- Senior Engineer Coding: guard clauses, named constants, edge case handling
- Error Handler: retry with jitter (2 attempts), structured fallback chain
- OWASP: no secrets in code, validated input, no injection vectors
- Idempotent installer: backup before write, skip existing, safe to re-run

10 · REFERENCE

Quick Reference

Key Paths

What	Path
Skills	~/.config/opencode/skills/
CLAUDE.md	~/.claude/CLAUDE.md
Memory data	~/.shokunin/memory/
Session logs	~/.shokunin/memory/sessions/
Backups	~/.shokunin/backups/
Current session	~/.shokunin/current-session.json
GitHub repo	github.com/EliasOulkadi/shokunin

Quick Commands

Action	Windows	Linux
Start session	opencode	opencode
Test memory	.\test-memory.ps1	./memory-healthcheck.sh
Git status	gst	gst
Git commit	ga + gc "msg"	ga + gc "msg"
npm install	ni	ni
Search memory	search-memory.ps1 -Query "X"	./search-memory.sh "X"

Version History

Version	Date	Changes
v1.0	2025.Q4	Initial 29 skills
v2.0	2026.Q1	All skills rewritten
v3.0	2026.04	Memory MCP, subagents, superpowers plugin
v4.0	2026.05.14	Memory rewrite: structured data, chroma-helper
v4.1	2026.05.14	Checkpoint timer, CI, profile shadow
v4.2	2026.05.14	Linux port: cross-platform paths, bash scripts

> Links
> Repo: github.com/EliasOulkadi/shokunin
> Install (Windows): `irm https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex`
> Install (Linux): `bash <(curl -sL https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)`
> License: MIT · Zero cost · Open source
> Created by Elias Oulkadi